

This is all you need to do for the author detail page to start working! Django calls the `'get_slug_field'` method here to get the name of the slug field in our model. Since the user model doesn't have such a thing we just help Django out a little in looking for the right thing.

The Tag Cloud View

In order to size different tags based on how many articles they appear in, we need to construct a query that includes this data in before sending it to the template. So in the end this particular pickle is all about finding the right queryset to get the desired results.

Django's ORM has exactly such a feature, called annotations, but it isn't something we have covered in the 'Django Models' chapter. The annotations feature allows us to add additional fields to each item in a queryset that are aggregated values calculated from fields of that model. For instance, if we had a model that had two numeric fields, `'value1'` and `'value2'`, we could use annotations while constructing a query to add in another field `'value_diff'` that is the difference between these fields.

Our case is a little different though, we want to add to each tag a count of the number of articles that have that tag applied to it. The query for this would go as follows:

- `from django.db.models import Count`
- `Tag.objects.annotate(num_articles=Count('article'))`

This will return a list of tags where each tag has an additional field called `'num_articles'` which is the number of articles that have that tag. We can now use this as the queryset of the TagList view and use the `'article_count'` value in the template to change the font sizes of the different tags.

Gallery Detail View

We wanted to be fancy and give users the ability to select one of three layouts for our gallery page. To actually implement this we have many different options, but they all start with one decision: do we want to have the view select which template to render, or do we want to have a single template that has all three layouts (within the same file or split out) and selects the layout in the template.

Our preference is to keep as little logic and code in the templates as possible, so our preference will be to have three different templates and have the view select which template to render based on the layout selected by the user.